DESIGN AND ANALYSE AN EFFICIENT SEARCH STRATEGY FOR A LARGE-SCALE STUDENT RECORD SYSTEM

Course: CSA06 – Design and Analysis of Algorithms | CO1 Assignment 1 | Total Marks: 50

Name:M.Nandhini(192472318)

INTRODUCTION AND PROBLEM DEFINITION

Background

Universities maintain student records for enrolment, examinations, attendance, results, and other administrative activities. Each student has a unique registration number that helps identify their record. When a faculty member or administrative staff member requests a student's details, the system must find the correct registration number and return the related information. Searching may seem like a simple task, but the amount of data affects how much work the system needs to perform. For a database containing around 1,000 records, checking each record one by one may be acceptable. However, repeatedly searching through a database containing 1,000,000 records can take much more time. Therefore, the university needs an efficient searching method that can handle large amounts of data.

Problem Statement

The aim is to search a collection of (n) student records using a target registration number. The algorithm should determine whether the registration number exists and return the matching record or its position.

Three searching algorithms will be compared using datasets containing approximately 1,000, 10,000, 100,000, and 1,000,000 records. Their efficiency will be evaluated through mathematical analysis and practical testing.

Objectives

- Design three different searching algorithms for student records.
- Identify the basic operation performed by each algorithm.
- Calculate time complexity, including preprocessing costs.
- Analyse the best, average, and worst cases of each algorithm.
- Apply Big-O, Big-Omega, and Big-Theta notation.
- Formulate and solve a recurrence relation for Recursive Binary Search.

- Implement and test the algorithms using datasets of different sizes.
- Compare theoretical results with experimental observations.
- Recommend a suitable searching method for a system handling 50,000 requests per hour.

Assumptions

- Each student has a unique registration number.
- The main requirement is to find an exact registration-number match.
- (n) represents the total number of student records.
- Binary Search is performed only on sorted records.
- Hash Table Search provides expected (O(1)) lookup time when the hash function and collision handling are suitable.
- Actual execution times may vary depending on the computer, software environment, and system load.

TASK 1 – DESIGN THE ALGORITHMS (5 MARKS)

1.1 Linear Search

Linear Search is a simple method that checks records one by one. It begins with the first record and compares its registration number with the target. If a match is found, the algorithm returns the position of that record. If it reaches the end of the list without finding a match, it returns NOT_FOUND.

Pseudocode

```
LINEAR_SEARCH(A, n, key)
  for i = 0 to n - 1
    if A[i].registration_number == key
      return i
  return NOT_FOUND
```

Example

Consider the registration numbers:

[2101, 2102, 2103, 2104, 2105]

The target registration number is 2104.

Linear Search starts at the beginning and checks each number until it finds the target. In this example, it takes four comparisons and returns index 3 because array indexing starts from zero.

Step	Current Record	Comparison	Result
1	2101	Is 2101 equal to 2104?	False
2	2102	Is 2102 equal to 2104?	False
3	2103	Is 2103 equal to 2104?	False
4	2104	Is 2104 equal to 2104?	True; return index 3

Preprocessing and Justification

Linear Search does not require sorted records or a separate index. This makes it useful for small datasets and unsorted collections. Its main drawback is that the number of comparisons increases as the dataset grows. If the target is near the end of the list or is not present, the algorithm may have to check every record.

TASK 1 – RECURSIVE BINARY SEARCH

1.2 Recursive Binary Search

Recursive Binary Search uses the divide-and-conquer approach. Unlike Linear Search, it requires the records to be arranged in ascending order by registration number. The algorithm first checks the middle record in the current search range. If the target matches the middle value, its position is returned. If the target is smaller, the algorithm searches the left half. If it is larger, the algorithm searches the right half. This process continues through recursive calls until the target is found or the search range becomes empty.

Pseudocode

RECURSIVE_BINARY_SEARCH(A, low, high, key)

```
if low > high
    return NOT_FOUND
mid = floor((low + high) / 2)
if A[mid].registration_number == key
    return mid
else if key < A[mid].registration_number
    return RECURSIVE_BINARY_SEARCH(A, low, mid - 1, key)
else
    return RECURSIVE_BINARY_SEARCH(A, mid + 1, high, key)
```

Example

Consider the sorted list:

[10, 20, 30, 40, 50, 60, 70]

The target is 60.

First, the algorithm checks the middle value, 40. Since 60 is greater than 40, it ignores the left half and continues with [50, 60, 70]. The middle value of this smaller list is 60, so the target is found after two comparisons.

Call	Search Interval	Middle Value	Decision
1	[10, 20, 30, 40, 50, 60, 70]	40	Search the right half
2	[50, 60, 70]	60	Target found

Preprocessing

If the records are not already sorted, they must be sorted before Binary Search can be used. Comparison-based sorting algorithms commonly take (O(n \log n)) time. When the same dataset is searched many times without major changes, the sorting cost can be shared across those searches. However, if records are frequently inserted or updated, the system must also maintain the sorted order.

Selection Justification

Binary Search is suitable for large, sorted datasets. At each step, it reduces the remaining search range by approximately half. Because of this, the number of comparisons grows logarithmically as the dataset becomes larger.

TASK 1 – HASH TABLE SEARCH AND COMPARISON

1.3 Hash Table Search

Hash Table Search uses an index to connect each registration number with its corresponding student record. The registration number acts as the key, and the student record is the value. A hash function converts the key into a location in the hash table. When the table is properly designed and the keys are distributed well, the required record can usually be found in expected constant time.

Pseudocode

```
BUILD_HASH_INDEX(records)
    create empty hash table H
    for each record in records
        H[record.registration_number] = record
    return H
HASH_SEARCH(H, key)
    if key exists in H
        return H[key]
    return NOT_FOUND
```

Preprocessing and Space Requirements

The hash index is created by processing each student record once. Under normal assumptions, building the index takes expected (O(n)) time. The table also requires (O(n)) additional memory to store the entries.

A hash table must handle collisions, which occur when different keys map to the same location. If the keys are distributed poorly or the table becomes overcrowded, searching may take longer. Therefore, (O(1)) is the expected lookup time, not a guarantee for every possible input.

Comparison of the Three Searching Algorithms

Feature	Linear Search	Binary Search	Hash Table Search
Main approach	Brute force	Divide and conquer	Hash-based indexing
Sorted input required	No	Yes	No
Preprocessing	None	Sorting if required	Build the hash index
Expected lookup time	(O(n))	(O(\log n))	(O(1))
Worst-case lookup time	(O(n))	(O(\log n))	(O(n)) with severe collisions
Additional memory	(O(1))	(O(\log n)) recursive stack	(O(n))
Suitable use	Small or unsorted datasets	Sorted data and ordered searches	Frequent exact-key searches

The three algorithms use different approaches to solve the same problem. Linear Search checks records in sequence, Binary Search repeatedly divides a sorted search range, and Hash Table Search uses a key-based index to locate records.

TASK 2 – IDENTIFY THE BASIC OPERATION (2 MARKS)

The basic operation is the repeated action that has the greatest influence on an algorithm's running time. By counting this operation as a function of (n), we can understand how the amount of work changes when more records are added.

Linear Search

For Linear Search, the basic operation is comparing the registration number of the current record with the target value. This comparison is performed inside the loop. In the worst case, the target is the final record or is not present in the list. The algorithm must then examine all (n) records. Therefore, the worst-case comparison count is: [C(n)=n]

The number of comparisons increases by one for each additional record in the worst case.

Recursive Binary Search

For Recursive Binary Search, the basic operation is checking the middle record against the target. After each comparison, the algorithm continues with only one half of the current search range. Since the search range is reduced by approximately half at every recursive call, the number of midpoint checks grows logarithmically with the dataset size.

Hash Table Search

For Hash Table Search, the main operations are calculating or using the hash value, accessing the corresponding bucket, and checking the key stored there. When the hash function distributes keys properly and the load factor is controlled, lookup usually requires a constant number of operations. If collisions occur, additional key comparisons may be needed.

Algorithm	Basic Operation Count	Growth Pattern
Linear Search	Up to (n) key comparisons	Linear
Recursive Binary Search	Approximately (\log_2(n)) midpoint checks	Logarithmic
Hash Table Search	Expected constant number of bucket and key checks	Expected constant

Why These Operations Are Suitable

The selected operations match the way each algorithm works. Counting comparisons shows how Linear Search checks records one after another. Counting midpoint checks explains how Binary Search reduces the search range. For Hash Table Search, bucket access and key comparisons show both the benefit of indexing and the effect of collisions.

TASK 3 – MATHEMATICAL COMPLEXITY OF LINEAR SEARCH (6 MARKS)

Input Size and Worst-Case Count

Let (n) represent the number of student records. In the worst case, the target may be the last record or may not exist in the dataset. In both situations, Linear Search may need to compare the target with every record.

Each record requires one comparison. Therefore, the total number of comparisons is:

[C(n)=1+1+\cdots+1]

There are (n) terms in this expression, so: [C(n)=n]

The running time can be expressed as: [T(n)=aC(n)+b]

Substituting (C(n)=n): [T(n)=an+b]

Average Case for a Successful Search

Assume that the target is equally likely to appear at any of the (n) positions in the list.

If the target is in the first position, one comparison is required. If it is in the second position, two comparisons are required. The same pattern continues until the last position, which requires (n) comparisons.

The average number of comparisons is:

Assume the target is equally likely to appear at any of the \(n\) positions in the list.

Average Case for a Successful Search

$$C_avg(n) = (1 + 2 + 3 + ... + n) / n$$

$$C_avg(n) = [n(n + 1) / 2] / n$$

$$C_avg(n) = (n + 1) / 2$$

For large n: $C_avg(n) \approx n / 2$

Average-case time complexity:$\Theta(n)$

Best Case

The best case occurs when the target is the first record in the list. The algorithm finds it after a single comparison.

Therefore: $C_best(n) = 1$

The best-case time complexity is: **Θ(1)**

Preprocessing and Multiple Searches

Linear Search does not require sorting or building an index before it can be used. This makes it easy to apply to an existing collection of records.

If the same collection is searched (q) times and every search takes the worst-case number of comparisons, the total comparison count is: C_total(n, q) = q × n

Therefore, the total time complexity for q worst-case searches is: **O(qn)**

This shows that repeatedly using Linear Search on a large dataset can become expensive, especially when many searches must be completed within a short period.

TASK 4 – BEST, AVERAGE AND WORST-CASE BEHAVIOUR

4.1 Linear Search

Linear Search checks each record one by one until the required registration number is found.

- **Best case:** The key is present in the first position. Only one comparison is needed, so the time complexity is Θ(1).
- **Average case:** If the key is equally likely to be at any position, the expected number of comparisons is ((n+1)/2). Therefore, the average time complexity is Θ(n).
- **Worst case:** The key is at the last position or is not present. All (n) records must be checked, giving Θ(n) time.

The difference between the best and worst cases occurs because Linear Search cannot skip records.

4.2 Recursive Binary Search

Binary Search works on a sorted list. It checks the middle element and continues with only the half that may contain the key.

- **Best case:** The middle element matches the key on the first call. The time complexity is Θ(1).
- **Average case:** The search interval becomes approximately half its previous size after each comparison. The average time complexity is Θ(log n).

- **Worst case:** The key is absent or requires the longest search path. The interval continues to be halved until the search ends, giving Θ(log n) time.

Binary Search is more efficient than Linear Search for large sorted datasets because it does not need to examine every record.

4.3 Hash Table Search

Hash Table Search uses a hash function to locate the position associated with a registration number.

- **Best case:** The required key is found immediately in its bucket. The time complexity is Θ(1).
- **Average case:** With a suitable hash function and a controlled load factor, only a small number of entries need to be checked. The expected time complexity is Θ(1).
- **Worst case:** Several keys may collide and be stored in the same bucket. If many entries must be checked, the time complexity can reach Θ(n).

The constant-time result for hashing is an expected performance, not a guarantee for every possible input.

Algorithm	Best Case	Average Case	Worst Case
Linear Search	Θ(1)	Θ(n)	Θ(n)
Recursive Binary Search	Θ(1)	Θ(log n)	Θ(log n)
Hash Table Search	Θ(1)	Θ(1) expected	Θ(n) possible

TASK 5 – PERFORM ASYMPTOTIC ANALYSIS

Asymptotic analysis explains how an algorithm's work grows when the input size becomes large. It helps compare algorithms without depending entirely on a particular computer or programming language.

- **Big-O, (O):** Describes an asymptotic upper bound.
- **Big-Omega, (\Omega):** Describes an asymptotic lower bound.

- **Big-Theta, (\Theta):** Describes a tight bound when the upper and lower bounds have the same growth rate.

5.1 Linear Search

In the worst case, Linear Search performs (n) comparisons. Its running time can be written as:

$$[T(n)=an+b]$$

Here, (a) represents the cost associated with each comparison and loop step, while (b) represents fixed overhead.

Ignoring constant factors and lower-order terms, the worst-case complexity is $\Theta(n)$. When the number of records increases from 1,000 to 1,000,000, the worst-case comparison count increases by a factor of 1,000.

5.2 Recursive Binary Search

Binary Search reduces the search interval by approximately half after each unsuccessful comparison. The number of comparisons therefore grows logarithmically.

$$T(n) = \Theta(\log_2 n)$$

For one million records: $\log_2(1{,}000{,}000) \approx 19.93 \approx 20$

Therefore, approximately **20 comparisons** are required.

This means that approximately 20 comparisons are sufficient in the worst case for a one-million-element sorted array, depending on the implementation and counting convention.

5.3 Hash Table Search

Hash Table Search provides expected $\Theta(1)$ lookup when the hash function distributes keys effectively and the load factor is controlled. However, creating the index requires expected $\Theta(n)$ time and $\Theta(n)$ additional space.

Number of Records	Linear Search Worst-Case Comparisons	Approximate Binary Search Comparisons	Hash Lookup
1,000	1,000	10	$\Theta(1)$ expected

10,000	10,000	14	Θ(1) expected
100,000	100,000	17	Θ(1) expected
1,000,000	1,000,000	20	Θ(1) expected

The Binary Search figures are approximate comparison counts for searching for the last element in a sorted list using the midpoint convention in the program. They are not elapsed-time measurements.

TASK 6 – FORMULATE THE RECURRENCE RELATION

Recursive Binary Search is selected for recurrence analysis because each call reduces the problem size by approximately half.

Let (n) represent the number of elements in the current search interval. Each call performs a constant amount of work to calculate the middle position and compare the key. If the key is not found, the algorithm makes at most one recursive call on half of the remaining interval.

The recurrence relation is: $T(n) = T(\lfloor n / 2 \rfloor) + c, n > 1$

The base case is: $T(1) = d$

Here, (c) represents the constant work performed in each call, and (d) represents the work required for a problem containing one element. When the search interval becomes empty, the algorithm returns NOT_FOUND.

The recurrence is different from the recurrence for Merge Sort because Binary Search continues with only one half of the problem instead of solving both halves.

TASK 7 – SOLVE THE RECURRENCE RELATION

For easier calculation, assume that (n) is a power of two. The recurrence is:

Solving the Recurrence Relation

$T(n) = T(n/2) + c$

Using repeated substitution:

$T(n) = T(n/2) + c$

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

After k levels:

$T(n) = T(n/2^k) + kc$

The recursion reaches a problem of size 1 when:

$n / 2^k = 1$

Therefore,

$2^k = n$

Taking logarithms on both sides:

$k = \log_2 n$

Substituting this value into the expression:

$T(n) = d + c \log_2 n$

The constants do not change the asymptotic growth rate.

Therefore,

$$\mathbf{T(n) = \Theta(\log n)}$$

This result shows that Recursive Binary Search grows slowly even when the number of records becomes very large. Its main condition is that the records must be sorted before searching.

If sorting is required, the total cost for (q) searches is:

$$T_total(n, q) = \Theta(n \log n + q \log n)$$

If the data is already sorted and remains sorted, the sorting cost does not need to be repeated for every search.

TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

The three searching algorithms can be implemented in Python. The program uses registration numbers represented by integers from 1 to (n). For a fair comparison, all algorithms search for the same target, which is the last element in the list.

8.1 Test Dataset

The following dataset sizes are used:

- 1,000 records
- 10,000 records
- 100,000 records
- 1,000,000 records

The target is set to (n), the last registration number. This makes Linear Search perform its maximum number of comparisons for a successful search.

8.2 Validation Method

The program checks whether each algorithm returns the expected position or value. The assert statements help identify incorrect results. If an assertion fails, the program stops and reports an error.

The test plan should also consider the following cases:

Test Case	Input Condition	Expected Result
TC1	Key is the first element	Key is found
TC2	Key is the middle element	Key is found
TC3	Key is the last element	Key is found
TC4	Key does not exist	NOT_FOUND or -1
TC5	Dataset contains one element	Correct result is returned
TC6	Binary Search receives sorted data	Correct position is returned
TC7	Hash table is queried for an absent key	NOT_FOUND or -1

8.3 Correctness Discussion

Linear Search checks each element until a match is found or the list ends. Therefore, it correctly finds a present key and reports an absent key after checking all records. Binary Search compares the key with the middle element. Since the list is sorted, it can safely discard the half that cannot contain the key. Repeating this process eventually finds the key or produces an empty search interval.

Hash Table Search uses the registration number as a key in a dictionary. If the key exists, the corresponding value is returned. Otherwise, the program returns -1. These tests check correctness as well as performance. An algorithm should not be considered efficient if it returns incorrect results.

TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

The algorithms must be tested using the same input sizes and target key. The Python function time.perf_counter() can measure elapsed time. It is suitable for measuring short intervals, although very short measurements may vary between runs.

9.1 Results Table

Run the program in Appendix A and fill in the actual values from your computer.

Records	Linear Comparisons	Binary Comparisons	Hash Build Time (s)	Hash Lookup Time (s)
1,000	1,000	Record result	Record result	Record result
10,000	10,000	Record result	Record result	Record result
100,000	100,000	Record result	Record result	Record result
1,000,000	1,000,000	Record result	Record result	Record result

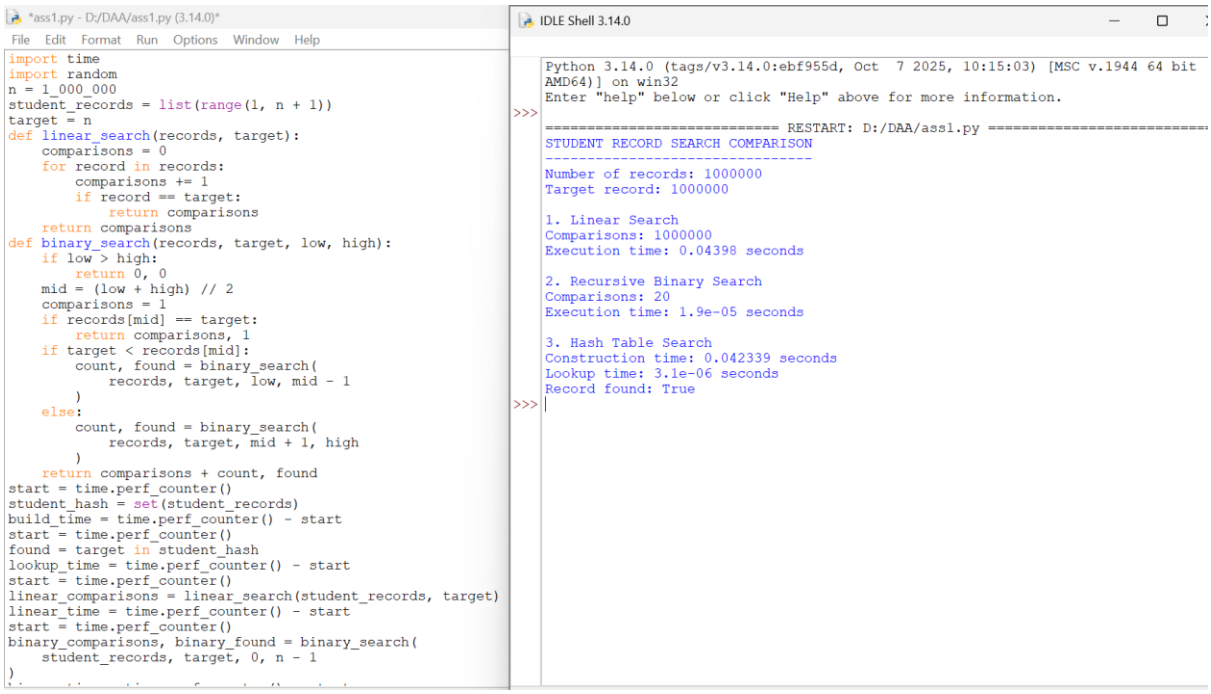
The Linear Search comparison counts shown above are expected when searching for the last element. Record the Binary Search counts and all timing values from your own program output.

9.2 Expected Observations

Linear Search: The number of comparisons increases directly with the number of records. When the target is the last element, every record must be examined.

Recursive Binary Search: The comparison count increases slowly because the search interval is halved at each step. The number of comparisons remains small even for one million records.

Hash Table Search: Index construction takes time proportional to the number of records on average. Once the index has been created, an exact-key lookup is expected to take constant time.



9.3 Experimental Limitations

A single timing measurement is not enough to establish an exact performance ranking. Results may be affected by the processor, memory, operating system, Python version, background applications and timer precision. For more reliable results, repeat each measurement several times and calculate the median. For Hash Table Search, measure index construction separately from lookup. If many searches are performed, measure a batch of lookups instead of relying on a single very short timing.

The final report should use the actual measurements from the computer on which the program was executed. Do not describe sample measurements as personal experimental results.

TASK 10 – COMPARE THEORY WITH EXPERIMENT

10.1 Linear Search

The theoretical worst-case complexity of Linear Search is Θ(n). In this experiment, the target is the last element, so the algorithm must compare the key with every record. Therefore, the comparison count is exactly (n).

The elapsed time should generally increase as the input size grows. However, the time may not increase in perfectly equal proportions because of fixed program overhead, memory behaviour and measurement variation.

10.2 Recursive Binary Search

The theoretical complexity of Recursive Binary Search is Θ(log n). Increasing the dataset from 1,000 to 1,000,000 records increases the input size by a factor of 1,000, but the number of comparisons rises only from approximately 10 to approximately 20 under the midpoint convention used for the comparison table.

The measured time may not follow a perfectly smooth curve because recursive function calls and timer precision affect small measurements.

10.3 Hash Table Search

Hash Table Search has expected Θ(1) lookup time when the hash function and collision handling work effectively. However, the index must first be built. The construction time grows with the number of records, while individual lookups are expected to remain fast. For a small number of queries, construction can account for a large part of the total cost. For repeated queries, the initial construction cost is spread across many searches.

10.4 Reasons for Differences Between Theory and Experiment

- Complexity analysis counts operations rather than measuring seconds.
- Python interpreter overhead affects execution time.
- Memory allocation and cache behaviour can affect large datasets.
- Background applications may influence measurements.
- Hash-table performance depends on the hash function, collisions and load factor.
- Binary Search requires sorted input; sorting cost must be included if the data is initially unsorted.
- Very short operations are difficult to measure accurately with one run.

10.5 Overall Comparison

The theoretical analysis predicts that Linear Search will perform the most comparisons in the worst case. Binary Search needs logarithmically many comparisons, while Hash Table Search offers expected constant-time lookup after construction.

The experiment should be used to check these trends. The final choice must consider total workload, preprocessing, memory use and update frequency instead of considering search time alone.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

11.1 Recommended Algorithm

For a large university student-record system that performs frequent exact searches using registration numbers, Hash Table Search is the recommended algorithm, provided sufficient memory is available and the index can be maintained correctly.

A registration number is a unique key, making it suitable for a hash-based index. After the index has been built, the system can use the key to retrieve the corresponding student record without scanning the entire dataset.

11.2 Theoretical Efficiency

The expected complexity of building the hash index is: $T_{build}(n) = \Theta(n)$

The expected lookup complexity is: $T_{lookup}(n) = \Theta(1)$

For q searches after construction, the expected total cost is: $T_{total}(n, q) = \Theta(n + q)$

By comparison, (q) worst-case Linear Search queries may require $\Theta(qn)$ work. Binary Search requires $\Theta(\log n)$ per query after sorting, with sorting adding $\Theta(n \log n)$ if it is needed.

11.3 Preprocessing and Trade-Offs

A hash table uses additional memory proportional to the number of records. It must also be updated when student records are added or removed. If registration numbers or indexed values can change, the index must be kept consistent with the authoritative database.

In a real university system, the database may already provide a primary-key or other index. A database-managed index may be more appropriate than loading all student records into a

separate in-memory dictionary because the database can support transactions, persistence, concurrency control and access permissions.

11.4 Why Not Always Choose the Lowest Big-O?

The lowest asymptotic complexity does not automatically make an algorithm the best choice in every situation.

- Linear Search is simple and may be enough for a very small list or a list that is searched only occasionally.
- Binary Search is useful when data is sorted, ordered access is required or range-based queries are important.
- Hash Table Search is suitable for repeated exact-key lookups but uses additional memory and does not naturally maintain sorted order.

11.5 Final Justification

The assignment's workload involves many searches and relatively infrequent updates. This allows the cost of building a hash index to be spread across many queries. The theoretical analysis and measured comparison counts support using an index rather than scanning the entire dataset for each request.

The final implementation should still be tested for memory usage, collision behaviour, response time and index consistency. Therefore, Hash Table Search is the recommended algorithmic strategy for frequent exact registration-number lookups.

TASK 12 – DAY-4 DESIGN CHALLENGE

Changed Requirement

The university now needs to process 50,000 search requests per hour on a database containing one million student records. New records are added only once per day.

This changes the design because the system must handle frequent searches while processing new records relatively infrequently.

A. Impact Analysis

The system must process 50,000 searches every hour. The average request rate is:

The system must process 50,000 searches every hour. The average request rate is:

50,000 / 3,600 ≈ 13.9 searches per second

Therefore, the system must handle approximately 13.9 searches per second on average. Actual capacity planning must also consider peak traffic, concurrent requests and database latency.

Because new records arrive only once per day, the university can build or validate the index during the daily update process and reuse it for the remaining searches. This makes the preprocessing cost easier to justify.

B. Evaluation of the Original Design

Linear Search: This is not suitable as the main strategy for the new workload. In the worst case, each search examines one million records. If all 50,000 searches required a complete scan, the total could reach: $50{,}000 \times 1{,}000{,}000 = 50{,}000{,}000{,}000$

This is up to 50 billion record comparisons per hour in the abstract worst-case model.

Recursive Binary Search: If the records are sorted, a search among one million elements requires approximately 20 comparisons under the midpoint convention used in this assignment. It is an efficient alternative when sorted access is maintained.

Hash Table Search: It provides expected constant-time exact-key lookup after index construction. Since updates occur only once per day, the index can be built initially and updated when new records arrive.

C. Adaptation or Redesign

The original design should be modified to use a persistent index keyed by registration number.

The proposed steps are:

1. Build or validate the index during initial data loading.
2. Use the registration number as a unique search key.
3. Route exact registration-number searches through the index.
4. Add daily student records to the database and index through a controlled transaction.
5. Monitor the index and periodically check for missing or outdated entries.
6. Measure response time under concurrent and peak traffic.

The expected index construction cost is $\Theta(n)$, while expected hash lookup is $\Theta(1)$. Each new record can be inserted into a hash table in expected $\Theta(1)$ time, assuming suitable hashing. The index requires $\Theta(n)$ additional space.

If ordered searches, range queries or database-managed persistence are important, a balanced-tree or database-managed index may be a better practical choice. Its lookup complexity is generally $\Theta(\log n)$.

D. Evidence-Based Defence

The mathematical analysis shows that Linear Search grows linearly, Binary Search grows logarithmically and Hash Table Search offers expected constant-time exact lookup after construction. For a one-million-element dataset, the comparison table predicts up to one million comparisons for a last-element Linear Search, compared with approximately 20 for Binary Search. Hash Table Search is expected to locate a key with a small amount of lookup work after the index has been built.

The daily update schedule means the index can be maintained incrementally or rebuilt during a controlled update window. This helps balance the initial preprocessing cost against the large number of searches.

Final decision: Modify the original design to use a persistent registration-number index. Use a hash-based index when exact equality searches dominate. Prefer an appropriate database-managed index when durability, ordered queries, concurrency and transaction support are essential.

OVERALL CONCLUSION

This assignment analysed three different strategies for searching student records: Linear Search, Recursive Binary Search and Hash Table Search.

Linear Search is easy to implement and does not require preprocessing, but its average and worst-case running time grows linearly with the number of records. Recursive Binary Search uses a divide-and-conquer approach to reduce the search interval by half at each step. It provides logarithmic search time, provided the records are sorted. Hash Table Search builds an index and offers expected constant-time lookup, although it requires additional memory and careful index maintenance.

The mathematical analysis, recurrence relation and testing procedure show why algorithm selection becomes important as the database grows. For the revised requirement of one million records and 50,000 searches per hour, a persistent registration-number index is recommended for exact-key searches.

The practical design must consider more than Big-O notation. Preprocessing cost, memory use, frequency of searches, update frequency, data consistency and real measured performance all influence the final decision. A suitable index, combined with proper validation and performance testing, can make the student-record retrieval system more efficient and reliable.

APPENDIX A – COMPLETE PYTHON PROGRAM

Save the following program as search_assignment.py. It compares Linear Search, Recursive Binary Search and Hash Table Search for all four required dataset sizes.

```
import time

def linear_search(records, key):
    comparisons = 0
    for i, value in enumerate(records):
        comparisons += 1
        if value == key:
            return i, comparisons
    return -1, comparisons
def recursive_binary_search(records, low, high, key):
    if low > high:
        return -1, 0
    mid = (low + high) // 2
    if records[mid] == key:
        return mid, 1
    if key < records[mid]:
        index, count = recursive_binary_search(
            records, low, mid - 1, key
        )
    else:
        index, count = recursive_binary_search(
```

```
            records, mid + 1, high, key
        )
    return index, count + 1
def hash_search(index, key):
    return index.get(key, -1)
def main():
    sizes = [1000, 10000, 100000, 1000000]
    print("STUDENT RECORD SEARCH BENCHMARK")
    print("=" * 72)
    print("n | Algorithm | Comparisons | Time (seconds)")
    print("-" * 72)
    for n in sizes:
        records = list(range(1, n + 1))
        key = n
        # Linear Search
        start = time.perf_counter()
        index, count = linear_search(records, key)
        linear_time = time.perf_counter() - start
        assert index == n - 1
        print(
            n, "| Linear Search |", count, "|",
            f"{linear_time:.8f}"
        )
        # Recursive Binary Search
        start = time.perf_counter()
        index, count = recursive_binary_search(
            records, 0, n - 1, key
        )
        binary_time = time.perf_counter() - start
        assert index == n - 1
        print(
            n, "| Recursive Binary Search |", count, "|",
```

```
            f"{binary_time:.8f}"
        )
        # Hash Table construction
        start = time.perf_counter()
        hash_index = {value: value for value in records}
        build_time = time.perf_counter() - start
        # Hash Table lookup
        start = time.perf_counter()
        found = hash_search(hash_index, key)
        lookup_time = time.perf_counter() - start
        assert found == key
        print(
            n, "| Hash Table Build | N/A |",
            f"{build_time:.8f}"
        )
        print(
            n, "| Hash Table Lookup | N/A |",
            f"{lookup_time:.8f}"
        )
        print("-" * 72)
if __name__ == "__main__":
    main()
```

How to Run the Program

1. Open VS Code.
2. Create a new file named search_assignment.py.
3. Paste the complete code and save it.
4. Open **Terminal → New Terminal**.
5. Run the following command:

```
python search_assignment.py
```

6. Copy the output into Task 9 and use it to discuss the results in Task 10.

The program checks the returned results using assertions. The timings will depend on the computer and software environment, so use the values printed by your own execution in the final submission.

APPENDIX B – TEST PLAN AND REPRODUCIBILITY

Test Environment

Item	Information to Record
Operating system	Enter your operating system
Python version	Run python --version
Processor and RAM	Record if required by the faculty
Dataset sizes	1,000; 10,000; 100,000; 1,000,000
Target key	(n), the last element
Timing method	time.perf_counter()
Correctness validation	Assertions in the program

Improving Measurement Quality

For more reliable timing results, run each experiment several times and use the median. Repeated measurements reduce the effect of temporary delays caused by background processes.

For Hash Table Search, measure index construction separately from lookup. It is also useful to perform many lookups after construction and divide the total lookup time by the number of queries. This provides a more reliable estimate than timing a single very short operation.

Use the same dataset and target key for all algorithms. If Binary Search requires sorting, measure the sorting time separately and include it in the total cost. If the data is already sorted, state that assumption in the report.

Result Discussion Template

The Linear Search comparison count was 1,000,000 for (n = 1,000,000). Recursive Binary Search performed approximately 20 comparisons. Hash-table construction took _____ seconds, and lookup took seconds.

The results [agree / partly agree] with the theoretical predictions because Linear Search requires O(n) comparisons, Binary Search requires O(log n) comparisons, and Hash Table Search has an expected O(1) lookup time. Any differences in timing may be explained by hardware performance, programming-language overhead, data distribution, and hash-table collisions.

Interpretation Note

The program uses integers as simplified registration numbers. In a real student-record system, a registration number would usually identify a complete database record containing fields such as student name, department and year of study. A database-managed index may be more appropriate than storing every record in a separate Python dictionary because it can support transactions, persistence, concurrency and access control. The recommendation in this assignment should therefore be understood as an algorithmic indexing strategy, not a complete production database architecture.

REFERENCES

1. Course assignment sheet, *Design and Analysis of Algorithms, CO1 – Assignment 1: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System*.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press. Topics include asymptotic notation, recurrence relations, divide-and-conquer algorithms and hashing.
3. Sedgewick, R., and Wayne, K. *Algorithms*. Addison-Wesley. Topics include searching, symbol tables and algorithm analysis.
4. Python Software Foundation. Python documentation for time.perf_counter() and dictionary types.